

Phát triển ứng dụng web

JavaScript
(cơ bản)

Nội dung

- ☐ HTML5 Semantic
- ☐ Giới thiệu JavaScript
- ☐ JavaScript – cơ bản
- ☐ JavaScript – HTML DOM
- ☐ JavaScript – events
- ☐ HTML Form và Validation

Nội dung

- ❑ **HTML5 Semantic**
- ❑ Giới thiệu JavaScript
- ❑ JavaScript – cơ bản
- ❑ JavaScript – HTML DOM
- ❑ JavaScript – events
- ❑ HTML Form và Validation

“Div-itis” – Vấn đề lạm dụng thẻ `<div>`

- ❑ Sử dụng CSS layout hiện đại => lạm dụng thẻ `<div>` cho mọi thứ trong cấu trúc trang web.

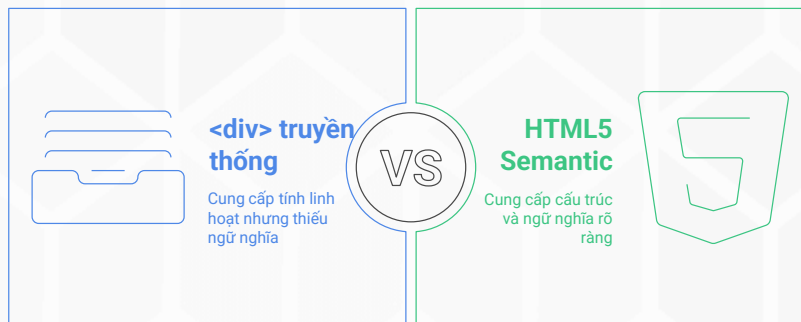
```
<div id="header">  
  <div class="logo">...</div>  
  <div class="nav">...</div>  
</div>  
<div id="content">  
  <div class="post">...</div>  
</div>  
<div id="sidebar">...</div>  
<div id="footer">...</div>
```

❑ Vấn đề:

- ❑ **Với máy móc (Trình duyệt, Google Bot):** không thể phân biệt các thẻ `<div>` quan trọng.
- ❑ **Với lập trình viên:** Khó đọc, khó bảo trì. Phải dựa hoàn toàn vào `id` và `class` để hiểu cấu trúc.

HTML Semantic là gì?

- ❑ Semantic = Relating to meaning (Liên quan đến ý nghĩa).
- ❑ **HTML Semantic** là việc sử dụng các thẻ HTML **đúng với ý nghĩa và chức năng** của nội dung mà nó bao bọc.



Cấu trúc trang với các thẻ Layout chính

- ❑ HTML5 cung cấp một bộ thẻ tiêu chuẩn để định hình các khối chính của một trang web.

Thẻ	Ý nghĩa
<header>	Khu vực đầu trang, thường chứa logo, slogan, thanh tìm kiếm, và điều hướng chính.
<nav>	Dành riêng cho các khối điều hướng chính của trang (menu chính, breadcrumbs).
<main>	Chứa nội dung độc nhất, chính yếu của trang. Mỗi trang chỉ nên có một thẻ <main>
<aside>	Chứa nội dung liên quan bên lề (sidebar, quảng cáo, thông tin bổ sung).
<footer>	Khu vực cuối trang, thường chứa thông tin bản quyền, liên hệ, sitemap.

Phân nhóm nội dung: <article> vs. <section>

- ❑ Đây là cặp thẻ dễ gây nhầm lẫn nhất.

 <article>	 <section>
Định nghĩa một mẫu nội dung hoàn chỉnh, độc lập và có thể được phân phối riêng rẽ.	Định nghĩa một khu vực, một nhóm nội dung có cùng chủ đề trong một trang.
Nội dung này có thể đăng lại trên một trang khác và vẫn có đầy đủ ý nghĩa không?	Thường sẽ có một tiêu đề (h1-h6) đi kèm để mô tả chủ đề của section đó.
Ví dụ: Một bài blog, một bình luận, một bài post trên forum.	Ví dụ: Phần "Giới thiệu", "Sản phẩm nổi bật", "Liên hệ" trên trang chủ.

Các thẻ ngữ nghĩa hữu ích khác

- ❑ **<figure>**: Dùng để bao bọc một media (hình ảnh, video, biểu đồ, đoạn code) được nhắc đến trong nội dung chính.

- ❑ **<figcaption>**: Cung cấp chú thích cho thẻ <figure>.

```
<figure>
  
  <figcaption>Hình 1.1 - Biểu đồ tăng trưởng người dùng quý 4.</figcaption>
</figure>
```

- ❑ **<time>**: Dùng để đánh dấu một ngày, giờ cụ thể. Giúp máy móc đọc và hiểu được thông tin thời gian.

- ❑ **datetime** attribute cung cấp định dạng chuẩn cho máy.

```
<p>Bài viết được xuất bản vào
  <time datetime="2025-10-05">ngày 5 tháng 10 năm 2025</time>.
</p>
```

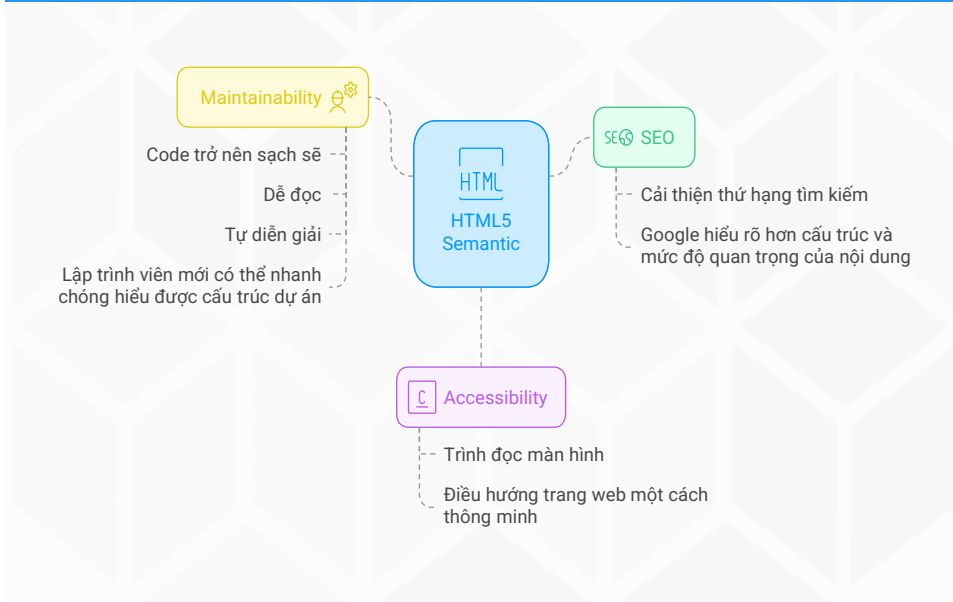
Kết quả

```
<div id="main-header">...</div>
<div id="main-content">
  <div class="blog-post">
    <h2>Tiêu đề bài viết</h2>
    <p>Nội dung...</p>
    <div class="post-footer">
      <span>Đăng ngày 5/10/2025</span>
    </div>
  </div>
</div>
```



```
<header>...</header>
<main>
  <article>
    <h2>Tiêu đề bài viết</h2>
    <p>Nội dung...</p>
    <footer>
      <p>Đăng vào <time datetime="2025-10-05">05-10-2025</time></p>
    </footer>
  </article>
</main>
```

Lợi ích khi sử dụng HTML5 Semantic



Nội dung

- ❑ *HTML5 Semantic*
- ❑ **Giới thiệu JavaScript**
- ❑ JavaScript – cơ bản
- ❑ JavaScript – HTML DOM
- ❑ JavaScript – events
- ❑ HTML Form và Validation

Giới thiệu về Script

- ❑ **Client-Side Script**
 - ❑ Script được thực thi tại *Client-Side* (trình duyệt): Thực hiện các tương tác với người dùng (tạo menu chuyển động, ...), kiểm tra dữ liệu nhập, ...
- ❑ **Server-Side Script**
 - ❑ Script được xử lý tại *Server-Side*, nhằm tạo các trang web có khả năng phát sinh nội dung động. Một số xử lý chính: kết nối CSDL, truy cập hệ thống file trên server, phát sinh nội dung html trả về người dùng ...

Giới thiệu về Javascript

- ❑ **JavaScript** Là ngôn ngữ **Client-side script** hoạt động trên trình duyệt của người dùng (*client*)
- ❑ **Chia sẻ xử lý** trong ứng dụng web. Giảm các xử lý không cần thiết trên server.
- ❑ Giúp **tạo các hiệu ứng, tương tác** cho trang web.

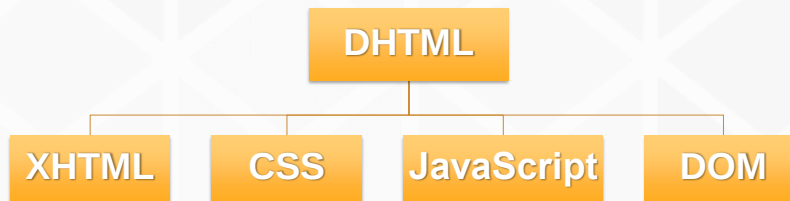


Nơi cung cấp

- ❑ Khi trình duyệt (*Client browser*) truy cập trang web có chứa các đoạn mã xử lý tại **server-side**. **Server (run-time engine)** sẽ thực hiện các lệnh **Server-side Scripts** và trả về nội dung **HTML** cho trình duyệt.
- ❑ Nội dung **HTML** trả về chủ yếu bao gồm: *mã html, client-script*.

Dynamic HTML (DHTML)

- ❑ Cho phép trang web có thể tương tác và thay đổi tùy theo hành động của người dùng.
- ❑ **DHTML** = **HTML** + **CSS** + **JavaScript**



15

Ý nghĩa JavaScript

- ❑ **HTML**: xác định nội dung trang web thông qua các thẻ ngữ nghĩa (**heading**, **paragraph**, **list**...).
- ❑ **CSS**: xác định luật hay định dạng để thể hiện tài liệu **HTML**
 - ❑ Font chữ
 - ❑ Nền (màu, hình ảnh, ...)
 - ❑ Vị trí và kích thước
- ❑ **JavaScript**: xác định các hành động
 - ❑ Tương tác thông qua các hành động của người dùng, xử lý sự kiện, ...

16

Nhúng JavaScript vào trang web

Định nghĩa **Script** trực tiếp trong trang **HTML**:

```
<script type="text/javascript">  
  <!--  
    // Lệnh Javascript  
  -->  
</script>
```

Nhúng sử dụng **script** cài đặt từ 1 file **.js** khác:

```
<script src="abc.js"></script>
```

Nhúng JavaScript vào trang Web

- ❑ **Web Browser** sẽ thực thi **<script>** khi load trang web theo thứ tự từ trên xuống dưới.
- ❑ Đối với **Source code JavaScript** có thể đặt trong các file **.js** sẽ được **nhúng** vào file **HTML** trước khi hoạt động.
- ❑ Các đoạn **code JavaScript** được **Browser** xử lý **cùng thứ tự** với các thẻ **HTML**. Trừ các **hàm (function)** chỉ được thực thi khi **có lời gọi hàm**.

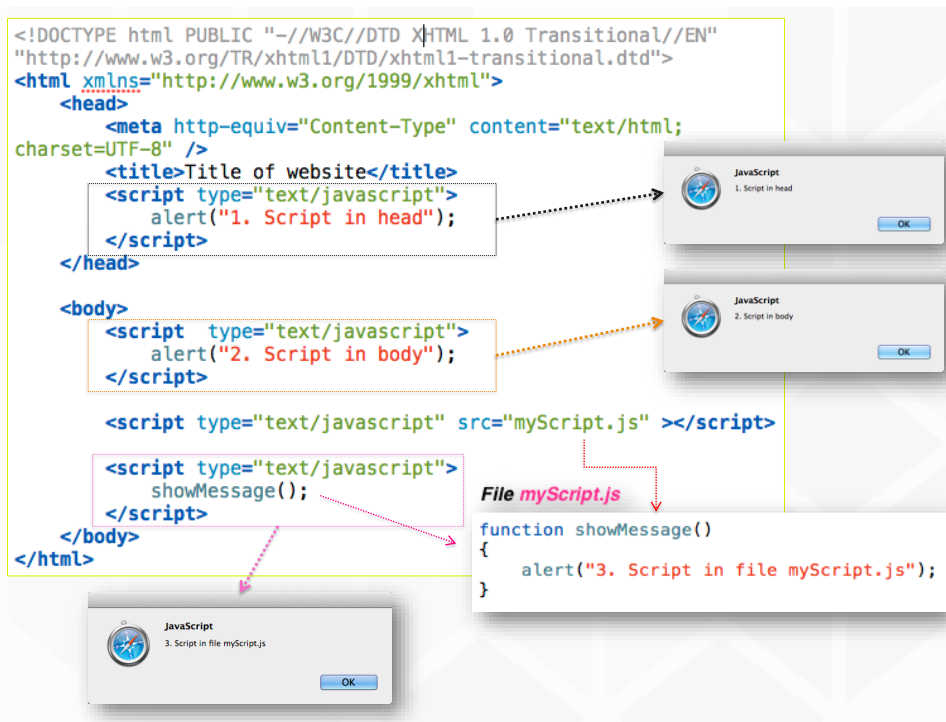
Nhúng JavaScript vào trang web

```
<html>
  <head>
    <script type="text/javascript">
      some statements
    </script>
  </head>
  <body>
    <script type="text/javascript">
      some statements
    </script>

    <script src="Tên_file_script.js">method()</script>
    <script type="text/javascript">
      // gọi thực hiện các phương thức được định nghĩa
      // trong "Tên_file_script.js"
    </script>
  </body>
</html>
```

Nhúng JavaScript vào trang web

- ☐ Đặt giữa tag **<head>** và **</head>**: **script** sẽ thực thi ngay khi trang web được mở.
- ☐ Đặt giữa tag **<body>** và **</body>**: **script** trong phần **body** được thực thi khi trang web đang mở (sau khi thực thi các đoạn script có trong phần **<head>**).
- ☐ Số lượng đoạn **client-script** chèn vào trang không hạn chế.



Nội dung

- ☐ HTML5 Semantic
- ☐ Giới thiệu JavaScript
- ☐ **JavaScript – cơ bản**
- ☐ JavaScript – HTML DOM
- ☐ JavaScript – events
- ☐ HTML Form và Validation

Biến trong JavaScript

❑ Cách đặt tên biến

- ❑ Bắt đầu bằng một chữ cái hoặc dấu _
- ❑ A..Z,a..z,0..9,_ : phân biệt HOA, Thường

❑ Khai báo biến

- ❑ Sử dụng từ khóa **var** (ít dùng), **let**, **const**

Ví dụ: ***let count = 10, amount;***

- ❑ *Không cần khai báo biến trước khi sử dụng*, biến thật sự tồn tại khi bắt đầu sử dụng lần đầu tiên.
- ❑ Biến *không cần phải khai báo kiểu dữ liệu* vì biến trong javascript *không có kiểu dữ liệu nhất định*

Kiểu dữ liệu trong Javascript

Kiểu dữ liệu	Ví dụ	Mô tả
Object	let listBooks = new Array(10);	Trước khi sử dụng, phải cấp phát bằng từ khóa new
String	"The cow jumped over the moon." "40"	Chuỗi ký tự, đặt trong dấu nháy đơn ' hoặc kép "
Number	0.066218 12	Số nguyên hoặc số thực. Theo chuẩn IEEE 754
boolean	true / false	
undefined	let myVariable;	Biến đã được khai báo nhưng chưa được gán giá trị. myVariable = undefined
null	connection.Close();	connection = null

Một biến trong JavaScript có thể lưu bất kỳ kiểu nào

Đổi kiểu dữ liệu

- ❑ Biến tự đổi kiểu dữ liệu khi giá trị mà nó lưu trữ thay đổi

Ví dụ:

```
let x = 10;           // x kiểu Number
x = "hello world!";  // x kiểu String
```

- ❑ Có thể cộng 2 biến khác kiểu dữ liệu

Ví dụ:

```
let x;
x = "12" + 34.5;      // KQ: x = "1234.5"
```

- ❑ Hàm **parseInt(...)**, **parseFloat(...)** : Đổi KDL từ chuỗi sang số.

Operators

- ❑ Toán tử số học: **+**, **-**, *****, **/**, **%** (chia lấy dư).
- ❑ Toán tử gán: **=**, **+=**, **-=**.
- ❑ Toán tử so sánh: **==** (so sánh giá trị), **===** (so sánh cả giá trị và kiểu), **!=**, **!==**, **>**, **<**.
- ❑ Toán tử logic: **&&** (AND), **||** (OR), **!** (NOT).

Hàm trong JavaScript

- ❑ Dạng thức khai báo chung:

```
function Tên_hàm(thamso1, thamso2, ...)  
{  
    ...  
}
```

- ❑ Hàm có giá trị trả về:

```
function Tên_hàm(thamso1, thamso2, ...)  
{  
    ...  
    return <value>;  
}
```



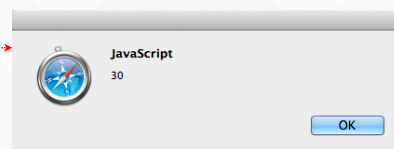
Hàm trong JavaScript

- ❑ Ví dụ:

```
function Sum(x, y)  
{  
    tong = x + y;  
    return tong;  
}
```

- ❑ Gọi hàm:

```
var x = Sum(10, 20);  
alert(x);
```



Các quy tắc chung

- ❑ Khối lệnh được bao trong dấu `{ }`
- ❑ Mỗi lệnh nên kết thúc bằng dấu `;`
- ❑ Cách ghi chú thích:
 - ❑ `//` *Chú thích 1 dòng*
 - ❑ `/*` *Chú thích*
nhiều dòng `*/`



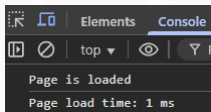
Phạm vi biến (Scope)

- ❑ **Global Scope:** Biến được khai báo bên ngoài hàm, có thể truy cập ở mọi nơi.
- ❑ **Function Scope:** Biến được khai báo bên trong hàm, chỉ có thể truy cập bên trong hàm đó.
- ❑ `let` và `const` có **phạm vi khối** (**Block Scope**), nghĩa là chỉ tồn tại trong `{ ... }` (như trong `if`, `for`, hoặc bất cứ khối nào). **Không thể khai báo lại** trong cùng phạm vi.
- ❑ `var` có **phạm vi hàm** (**Function Scope**). Có thể **khai báo lại** trong cùng phạm vi.

console.log() - Người bạn tốt nhất của Developer

- ❑ Công cụ để in thông tin ra tab Console của trình duyệt.
- ❑ Cực kỳ hữu ích để gỡ lỗi (debug) và kiểm tra giá trị của biến.

```
<script>
  let startDate = new Date();
  function pageLoaded() {
    console.log("Page is loaded");
    let currentDate = new Date();
    let loadTime = currentDate - startDate; // in milliseconds
    console.log("Page load time: " + loadTime + " ms");
  }
  pageLoaded();
</script>
```



Câu lệnh if

```
if (condition)
{
    statement[s] if true
}
else
{
    statement[s] if false
}
```

Ví dụ:

```
var x = 5, y = 6, z;
if (x == 5)
{
    if (y == 6)
        z = 17;
}
else
    z = 20;
```


Câu lệnh **switch**

```
switch (expression)  
{  
    case label1 :  
        statementlist  
    case label2 :  
        statementlist  
    ...  
    default :  
        statement list  
}
```

Ví dụ :

```
var diem = "G";  
switch (diem) {  
    case "Y":  
        document.write("Yếu");  
        break;  
    case "TB":  
        document.write("Trung bình");  
        break;  
    case "K":  
        document.write("Khá");  
        // break;  
    case "G":  
        document.write("Giỏi");  
        break;  
    default:  
        document.write("Xuất sắc");  
}
```

Vòng lặp **for**

```
for ([initial expression]; [condition]; [update expression])  
{  
    statement[s] inside loop  
}
```

Ví dụ:

```
var myarray = new Array();  
for (i = 0; i < 10; i++)  
{  
    myarray[i] = i;  
}
```

Vòng lặp `for...in`

```
for (key in object)
{
    statement[s] inside loop
}
```

Ví dụ:

```
const person = { fname: "A", lname: "Tran", age: 20 };
for (let e in person)
{
    //...
}_
```

Vòng lặp `for...of`

```
for (variable of iterable)
{
    statement[s] inside loop
}
```

Ví dụ:

```
const arr = [ "A", "B", "C" ];
for (let s of arr)
{
    //...
}_
```

Vòng lặp **while** & **do ... while**

```
while (expression)
{
    statements
}
```

Ví dụ:

```
var i = 9, total = 0;
while (i < 10)
{
    total += i * 3 + 5;
    i = i + 5;
}
```

```
do
{
    statements
} while (expression);
```

Ví dụ:

```
var i = 9, total = 0;
do
{
    total += i * 3 + 5;
    i = i + 5;
} while (i < 10);
```

Nội dung

- ☐ HTML5 Semantic
- ☐ Giới thiệu JavaScript
- ☐ JavaScript – cơ bản
- ☐ **JavaScript – HTML DOM**
- ☐ JavaScript – events
- ☐ HTML Form và Validation

Đối tượng HTML DOM

- ❑ **DOM** = Document Object Model
- ❑ Là một giao diện lập trình cho các tài liệu HTML. Trình duyệt tạo ra một cấu trúc cây của các đối tượng từ file HTML được dùng để **truy xuất** và **thay đổi thành phần HTML** trong trang web bằng **JavaScript** (thay đổi nội dung của trang).
- ❑ Một số đối tượng của DOM: *window, document, history, link, form, frame, location, event, ...*

Đối tượng Window - DOM

- ❑ Là thể hiện của đối tượng **cửa sổ trình duyệt**
- ❑ Tồn tại khi mở 1 tài liệu HTML
- ❑ Sử dụng để truy cập thông tin của các đối tượng trên cửa sổ trình duyệt (tên trình duyệt, phiên bản trình duyệt, thanh tiêu đề, thanh trạng thái ...)

Thành phần

❑ Properties

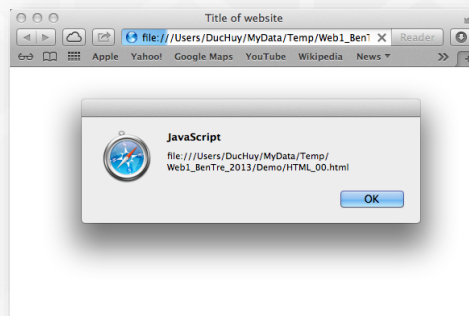
- ❑ document
- ❑ event
- ❑ history
- ❑ location
- ❑ name
- ❑ navigator
- ❑ screen
- ❑ status

● Methods

- Alert
- Confirm
- Prompt
- Blur
- close
- Focus
- open

Ví dụ

```
<html>
  <body>
    <script type="text/javascript">
      var curURL = window.location;
      window.alert(curURL);
    </script>
  </body>
</html>
```



Đối tượng Document - DOM

- ❑ Biểu diễn cho **nội dung trang HTML** đang được hiển thị trên trình duyệt. Là một đối tượng toàn cục đại diện cho toàn bộ tài liệu.
- ❑ Là điểm khởi đầu để truy cập vào mọi thứ trên trang. Dùng để lấy thông tin về tài liệu, các thành phần HTML và xử lý sự kiện



Thành phần

Properties

- aLinkColor
 - bgColor
 - body
 - fgColor
 - linkColor
 - title
 - URL
 - vlinkColor
 - forms[]
 - images[]
 - childNodes[]
- documentElement
cookie
.....

Methods

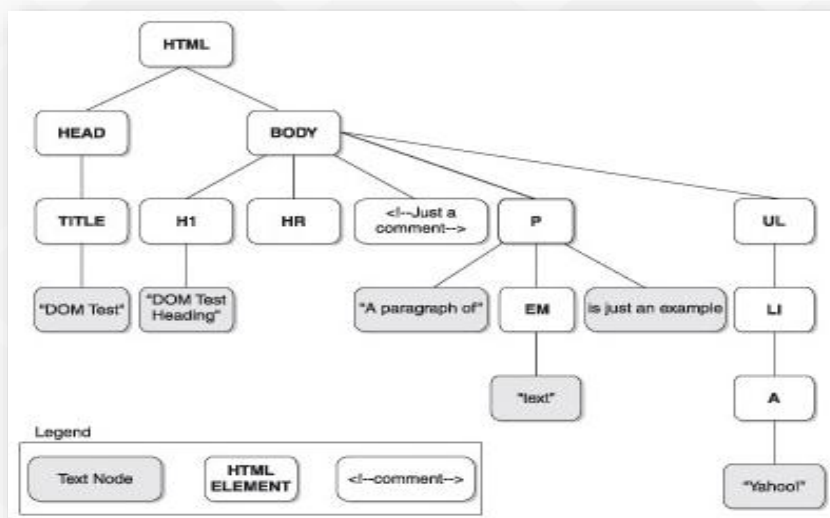
- close
- open
- createTextNode(" text ")
- createElement("HTMLtag")
- getElementById("id")
- ...

Minh họa

❑ Biểu diễn nội dung của tài liệu theo cấu trúc cây

```
<html>
  <head>
    <title>DOM Test</title>
  </head>
  <body>
    <h1>DOM Test Heading</h1>
    <hr />
    <!-- Just a comment -->
    <p id="p1">A paragraph of <em>text</em> is just an example</p>
    <ul>
      <li>
        <a href="http://www.yahoo.com">Yahoo! </a>
      </li>
    </ul>
  </body>
</html>
```

Cấu trúc cây nội dung tài liệu



Phân loại node

❑ Các loại **DOM Node** chính

Node Type Number	Loại	Mô tả	Vi dụ
1	Element	(X)HTML or XML element	...
2	Attribute	Thuộc tính của HTML hay XML element	align="center"
3	Text	Nội dung chứa trong HTML or XML element	This is a text fragment!
8	Comment	HTML comment	<!-- This is a comment -->
9	Document	Đối tượng tài liệu gốc, thường là element nằm ở cấp cao nhất trong cây cấu trúc của tài liệu	<html>
10	DocumentType	Định nghĩa loại tài liệu	<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">

Truy xuất phần tử DOM (Selecting Elements)

- ❑ **getElementById** ('id'): Lấy **một phần tử** có ID cụ thể (nhANH NHẤT).
- ❑ **getElementsByTagName** ('tagName'): Lấy **một danh sách** các phần tử theo tên thẻ.
- ❑ **getElementsByClassName** ('className'): Lấy **một danh sách** các phần tử theo tên lớp CSS.

Ví dụ:

```
<p id="id1" class="c1" >  
    some text  
</p>
```

```
const node = document.getElementById("id1");  
const nodes1 = document.getElementsByTagName("p");  
const nodes2 = document.getElementsByClassName("c1");
```


Phương thức truy xuất hiện đại

- ❑ **querySelector** ('selector'): Lấy **phần tử đầu tiên** khớp với **CSS selector**. Rất mạnh mẽ và linh hoạt.
- ❑ **querySelectorAll** ('selector'): Lấy **tất cả** các phần tử khớp với **CSS selector**.

Ví dụ:

```
<p id="id1" class="c1" >  
  some text  
</p>
```

```
const node = document.querySelector("#id1");  
const nodes1 = document.querySelectorAll("p");  
const nodes2 = document.querySelectorAll(".c1");
```

Thay đổi nội dung phần tử - HTML

❑ innerHTML

Thay đổi toàn bộ HTML bên trong phần tử.

Ví dụ:

```
//<p id="para1" >  
//  some text  
//</p>  
const theElement = document.getElementById("para1");  
theElement.innerHTML = "Some <b> new </b> text";
```

```
// Kết quả :  
// <p id="para1" >  
// Some <b> new </b> text  
// </p>
```

Thay đổi nội dung phần tử - Text

❑ `innerText`

Tương tự *innerHTML*, tuy nhiên an toàn hơn vì bất kỳ nội dung nào đưa vào cũng được xem như là text hơn là các thẻ HTML.

Ví dụ:

```
const theElement = document.getElementById("para1");  
theElement.innerText = "Some <b> new </b> text";
```

```
// Kết quả hiển thị trên trình duyệt  
// bên trong thẻ p: "Some <b> new </b> text"
```

Thay đổi Attributes và Style

❑ Thay đổi Attributes: `element.src`, `element.href`, `element.alt`,...

❑ Thay đổi Style: truy cập thành phần `style`.

❑ Ví dụ:

```
<a href="#">Click me!</a>  
<script>  
  const a = document.querySelector("a");  
  a.href = "https://www.example.com";  
  a.target = "_blank";  
  a.style.color = "red";  
  a.style.textDecoration = "none";  
</script>
```



Thêm và Xóa Class

❑ Sử dụng thuộc tính **classList** là cách tốt nhất.

- ❑ `element.classList.add('new-class')`
- ❑ `element.classList.remove('old-class')`
- ❑ `element.classList.toggle('active-class')`

Tạo node mới

❑ **createElement** (*nodeName*)

Cho phép tạo ra 1 node HTML mới tùy theo đối số *nodeName* đầu vào

Ví dụ:

```
const imgNode = document.createElement("img");  
imgNode.src = "images/test.gif";  
  
// 
```

Tạo Text node

❑ `createTextNode` (*content*)

Ví dụ:

```
const textNode = document.createTextNode("New text");
const pNode = document.createElement("p");
pNode.appendChild(textNode);

// <p>New text</p>
```

Thêm node mới vào tài liệu HTML

❑ `appendChild` (*newNode*)

Chèn node mới **newNode** vào cuối danh sách các node con của một node.

Ví dụ:

```
//<p id="id1" >
//    some text
//</p>
var pNode = document.getElementById( id1 );
var imgNode = document.createElement("img");
imgNode.src = "images/test.gif";
pNode.appendChild(imgNode);

//<p id="id1" > some text  </p>
```

Nội dung

- ☐ *HTML5 Semantic*
- ☐ *Giới thiệu JavaScript*
- ☐ *JavaScript – cơ bản*
- ☐ *JavaScript – HTML DOM*
- ☐ **JavaScript – events**
- ☐ HTML Form và Validation

Event là gì?

- ☐ Là các hành động xảy ra trên trang web mà JavaScript có thể "phản ứng" lại. Ví dụ: người dùng click chuột, di chuyển chuột, nhấn phím, gửi form...
 - ⇒ Đây là chìa khóa để tạo ra sự tương tác.
- ☐ **Event Listener**: Là một hàm sẽ được thực thi khi một sự kiện cụ thể xảy ra trên một phần tử.

Các sự kiện **thông dụng**

☐ Các sự kiện được hỗ trợ bởi hầu hết các đối tượng

- ☐ **onClick**
- ☐ **onFocus**
- ☐ **onChange**
- ☐ **onBlur**
- ☐ **onMouseOver**
- ☐ **onMouseOut**
- ☐ **onMouseDown**
- ☐ **onMouseUp**
- ☐ **onLoad**
- ☐ **onSubmit**
- ☐ **onResize**
- ☐ ...



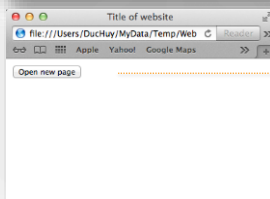
Xử lý sự kiện **cho các thẻ HTML**

☐ Cú pháp:

<TAG **eventHandler** = **"JavaScript Code"**>

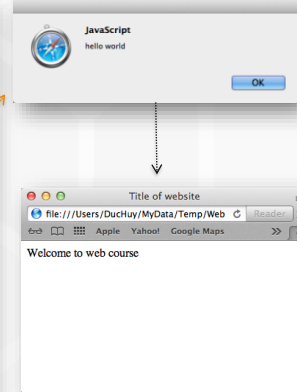
☐ Lưu ý: Dấu "..." và '...'

```
<body>
  <input type="button"
        name="btnClickMe"
        value="Open new page"
        onclick="window.open('http://www.google.com');" />
</body>
```



Xử lý sự kiện bằng function

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0  
Transitional//EN" "http://www.w3.org/TR/xhtml1/  
DTD/xhtml1-transitional.dtd">  
<html xmlns="http://www.w3.org/1999/xhtml">  
  <head>  
    <meta http-equiv="Content-Type"  
content="text/html; charset=UTF-8" />  
    <title>Title of website</title>  
    <script type="text/javascript">  
      function ShowMessage() {  
        alert("hello world");  
      }  
    </script>  
  </head>  
  <body onload="ShowMessage()">  
    Welcome to web course  
  </body>  
</html>
```



Xử lý sự kiện bằng thuộc tính

- ❑ Gán tên hàm xử lý cho 1 object event
object.eventhandler = function_name;

```
<html>  
  <head>  
    <script language="Javascript">  
      function GreetingMessage()  
      {  
        window.alert("Welcome to my world");  
      }  
      window.onload = GreetingMessage ();  
    </script>  
  </head>  
  <body>  
  </body>  
</html>
```

Đối tượng Event (The Event Object)

- ❑ Khi một sự kiện xảy ra, trình duyệt tạo ra một đối tượng chứa thông tin về sự kiện đó.
- ❑ Đối tượng này được tự động truyền vào hàm xử lý sự kiện (event handler).
- ❑ Cách xử lý hiện đại và tốt nhất:

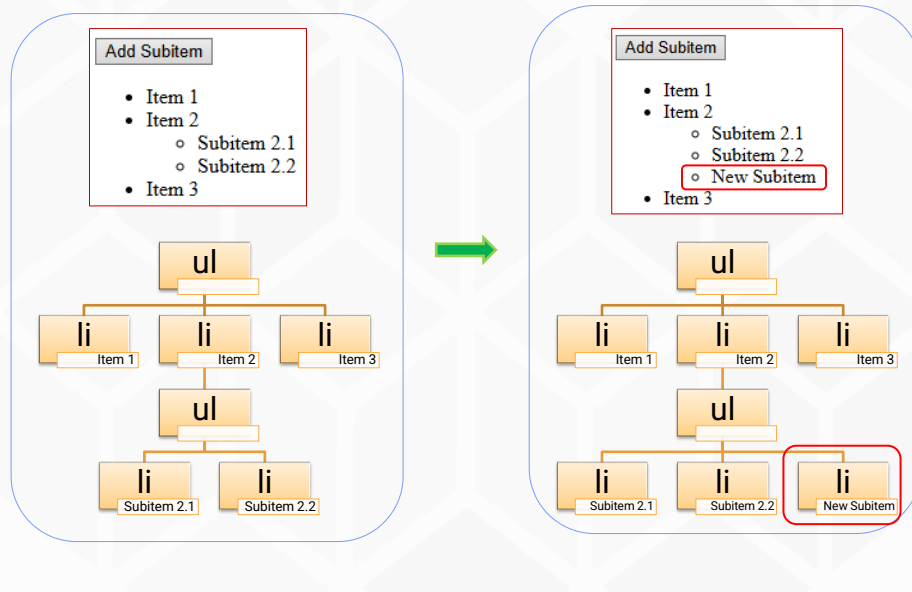
```
const button = document.querySelector('#myButton');
button.addEventListener('click', function (evt) {
  console.log('Button was clicked!');
  console.log(evt);           // Xem thông tin về sự kiện
  console.log(event.target);  // Phần tử đã gây ra sự kiện
});
```

Event Bubbling

- ❑ Khi một sự kiện xảy ra trên một phần tử, nó cũng xảy ra trên tất cả các phần tử cha của nó.

Phương thức	Tác dụng	Ảnh hưởng đến lan truyền	Ảnh hưởng đến hành động mặc định
<code>event.preventDefault()</code>	Ngăn hành động mặc định sự kiện	Không	Có
<code>event.stopPropagation()</code>	Ngăn sự kiện lan truyền tiếp tục (bubbling/capturing)	Có	Không
<code>event.stopImmediatePropagation()</code>	Ngăn sự kiện lan truyền + ngăn tất cả listener khác trên phần tử	Có	Không

Ví dụ thêm nội dung trang web



JS

```
const btn = document.querySelector('#addSI');
btn.addEventListener('click', function () {
  const ul = document.querySelector('ul li:nth-child(2) > ul');
  const li = document.createElement('li');
  li.textContent = 'New Subitem';
  ul.appendChild(li);
});
```

Hoặc sử dụng mã HTML

```
btn.addEventListener('click', function () {
  const ul = document.querySelector('ul li:nth-child(2) > ul');
  ul.insertAdjacentHTML('beforeend', '<li>New Subitem</li>');
});
```

Nội dung

- ☐ *HTML5 Semantic*
- ☐ *Giới thiệu JavaScript*
- ☐ *JavaScript – cơ bản*
- ☐ *JavaScript – HTML DOM*
- ☐ *JavaScript – events*
- ☐ **HTML Form và Validation**

HTML Form và Validation

- ☐ **HTML Form**
- ☐ Form Validation

Giới thiệu Form

- ❑ Là công cụ chính để thu thập dữ liệu từ người dùng.
- ❑ Giúp gửi yêu cầu của người dùng đến nơi xử lý trong ứng dụng web
- ❑ Tag **<form>** dùng để chứa các thành phần khác của form
- ❑ Những thành phần nhập liệu được gọi là **Form Field**
 - ❑ text field
 - ❑ password field
 - ❑ multiple-line text field
 - ❑

Tag <Form>

- ❑ Là container chứa các thành phần nhập liệu khác.

```
<form name="..." action="..." method="...">  
    <!-- các thành phần của Form -->  
</form>
```

- ❑ Các thuộc tính của **<form>**
 - ❑ **NAME** : tên FORM
 - ❑ **ACTION** : chỉ định trang web nhận xử lý dữ liệu từ FORM này khi có sự kiện click của button **SUBMIT**.
 - ❑ **METHOD** : Xác định phương thức chuyển dữ liệu (**POST,GET**)

Tag <Form> - Ví dụ

Dangnhap.html

```
<html>
  <body>
    <form name="Dangnhap"
          action="/admin/xlDangnhap.php"
          method="Post">
      .....
    </form>
  </body>
</html>
```

Các thành phần của Form

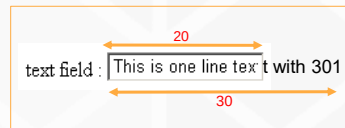
☐ Gồm các loại Form Field sau:

- ☐ Text field
- ☐ Password field
- ☐ Hidden Text field
- ☐ Check box
- ☐ Radio button
- ☐ File Form Control
- ☐ Submit Button, Reset Button, Generalized Button
- ☐ Multiple-line text field
- ☐ Label
- ☐ Pull-down menu
- ☐ Scrolled list
- ☐ Field Set

Text Field

- ❑ Dùng để nhập một dòng văn bản
- ❑ Cú pháp

```
<INPUT  
  TYPE           = "TEXT"  
  NAME           = string  
  READONLY  
  SIZE           = variant  
  MAXLENGTH      = long  
  TABINDEX       = integer  
  VALUE          = string  
  .....
```



- ❑ Ví dụ

```
<input type="text" name="txtName" value="This is one line text with  
301" size="20" maxlength="30">
```

Password Field

- ❑ Dùng để nhập mật khẩu
- ❑ Cú pháp

```
<INPUT  
  TYPE           = "PASSWORD"  
  NAME           = string  
  READONLY  
  SIZE           = variant  
  MAXLENGTH      = long  
  TABINDEX       = integer  
  VALUE          = string  
  .....
```



- ❑ Ví dụ

```
<input type="Password" name="txtPassword" value="123456abc1234"  
size="20" maxlength="30">
```

Hidden Text Field

- ❑ Dùng để truyền 1 giá trị của thuộc tính value khi form được submit
- ❑ **Không hiển thị** ra trên màn hình
- ❑ Cú pháp

```
<INPUT  
  TYPE           = "HIDDEN"  
  NAME           = string  
  READONLY  
  SIZE           = variant  
  MAXLENGTH     = long  
  TABINDEX      = integer  
  VALUE          = string  
  .....  
>
```

hidden text field :

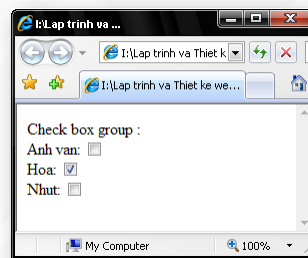
- ❑ Ví dụ : `hidden text field : <input type="hidden" name="txtHidden" value="This is hidden text.You can't see.">`

Check box

- ❑ Cú pháp

```
<input  
  TYPE           = "checkbox"  
  NAME           = "text"  
  VALUE          = "text"  
  [checked]  
>
```

- ❑ Ví dụ

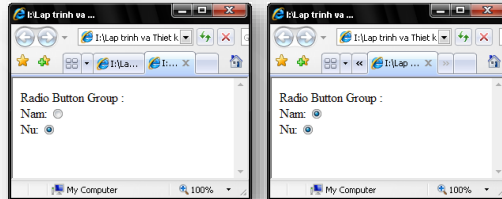


```
<html>  
  <body>  
    Check box group : <br>  
    Anh van: <input type="checkbox" name="Languages" value="En"><br>  
    Hoa: <input type="checkbox" name="Languages" value="Chz" checked><br>  
    Nhut: <input type="checkbox" name="Languages" value="Jp"><br>  
  </body>  
</html>
```

Radio button

Cú pháp

```
<input  
  TYPE = "radio"  
  NAME = "text"  
  VALUE = "text"  
  [checked]  
>
```



Ví dụ

```
<html>  
  <body>  
    Radio Button Group : <br>  
    Nam: <input type="radio" name="sex" value="nam" checked><br>  
    Nu: <input type="radio" name="sex" value="nu" checked><br>  
  </body>  
</html>
```

```
<html>  
  <body>  
    Radio Button Group : <br>  
    Nam: <input type="radio" name="sex1" value="nam" checked><br>  
    Nu: <input type="radio" name="sex2" value="nu" checked><br>  
  </body>  
</html>
```

File upload Control

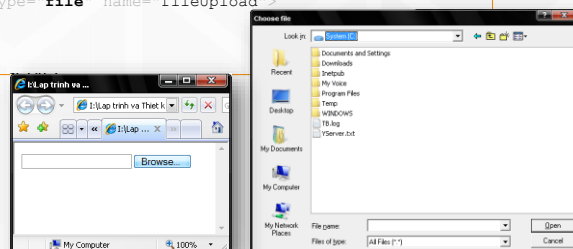
Dùng để upload 1 file lên server

Cú pháp

```
<form action="..." method="post" enctype="multipart/form-data"  
  name="...">  
  <input TYPE="FILE" NAME="...">  
</form>
```

Ví dụ

```
<html>  
  <body>  
    <form name="frmMain" action="POST" enctype="multipart/form-data">  
      <input type="file" name="fileUpload">  
    </form>  
  </body>  
</html>
```



Submit button

- ❑ Nút phát lệnh và gửi dữ liệu của form đến trang xử lý.
- ❑ Mỗi form chỉ có **duy nhất một nút submit** và nút này được viền đậm
- ❑ Cú pháp:

```
<input TYPE="submit" name="..." value="...">
```

- ❑ Lưu ý: *Luôn luôn lắng nghe sự kiện submit trên chính thẻ <form>, không phải trên nút bấm.*

Reset Button

- ❑ Dùng để trả lại giá trị mặc định cho các control khác trong form
- ❑ Cú pháp

```
<input TYPE="reset" name="..." value="...">
```

- ❑ Ví dụ

```
<input type="reset" name="btnReset" value="Rest">
```

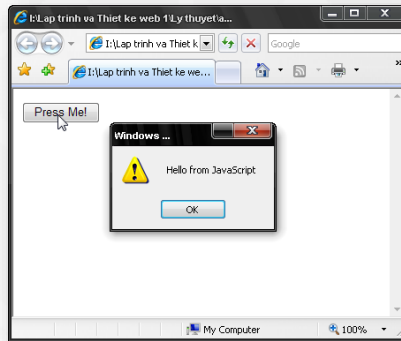

Generalized Button

❑ Cú pháp

```
<input type="button" name="..." value="..." onclick="script">
```

❑ Ví dụ

```
<input type="button" name="btnNormal" value="Press Me!"  
onclick="alert('Hello from JavaScript');">
```



Multiline Text Field

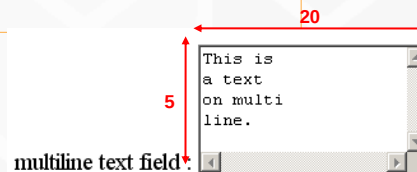
❑ Dùng để nhập văn bản nhiều dòng

❑ Cú pháp

```
<TEXTAREA  
  COLS           = long  
  ROWS           = long  
  DISABLED  
  NAME           = string  
  READONLY  
  TABINDEX       = integer  
  WRAP           = OFF | PHYSICAL | VIRTUAL> .....  
</TEXTAREA>
```

❑ Ví dụ

```
<textarea cols="20" rows="5"  
  wrap="off">  
  This is a text on multiline.  
</textarea>
```



Label

- ❑ Dùng để gán nhãn cho một Form Field

- ❑ Cú pháp

```
<LABEL  
  FOR = IDString  
  CLASS=string  
  STYLE=string  
>
```

- ❑ Ví dụ

Anh văn: ☐

```
<label for="Languages">Anh văn: </label>  
<input type="checkbox" name="Languages" id="Languages" value="Eng">
```

Pull-down Menu

- ❑ Dùng để tạo ra một ComboBox

- ❑ Cú pháp

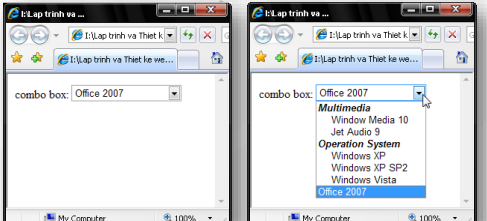
```
<Select name="...">  
  <optgroup label="...">  
    <option [selected] value="..." >.....</option>  
    .....  
  </optgroup>  
  
  <option [selected] value="..." >.....</option>  
  .....  
</select>
```

Pull-down Menu

```

<html>
<body>
  combo box:
  <select name="DSSoftware">
    <optgroup label="Multimedia">
      <option value="WM10">Window Media 10</option>
      <option value="JA9">Jet Audio 9</option>
    </optgroup>
    <optgroup label="Operation System">
      <option value="WXP">Windows XP</option>
      <option value="WXPSP2">Windows XP SP2</option>
      <option value="WVT">Windows Vista</option>
    </optgroup>
    <option selected value="Office07">Office 2007</option>
  </select>
</body>
</html>

```



Field Set

- ❑ Dùng để tạo ra GroupBox, nhóm các thành phần nhập liệu trong form

- ❑ Cú pháp

```

<fieldset>
  <legend>GroupBox's Name</legend>
  <input .....>
  ...
</fieldset>

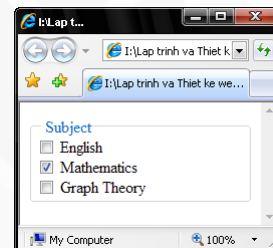
```

- ❑ Ví dụ

```

<html>
<body>
  <fieldset>
    <legend>Subject</legend>
    <input type="checkbox" name="Subjects" value="Eng"> English<br>
    <input type="checkbox" name="Subjects" value="Math" checked> Mathematics<br>
    <input type="checkbox" name="Subjects" value="GraphTheory"> Graph Theory<br>
  </fieldset>
</body>
</html>

```



Phương thức GET/POST

HTML Form

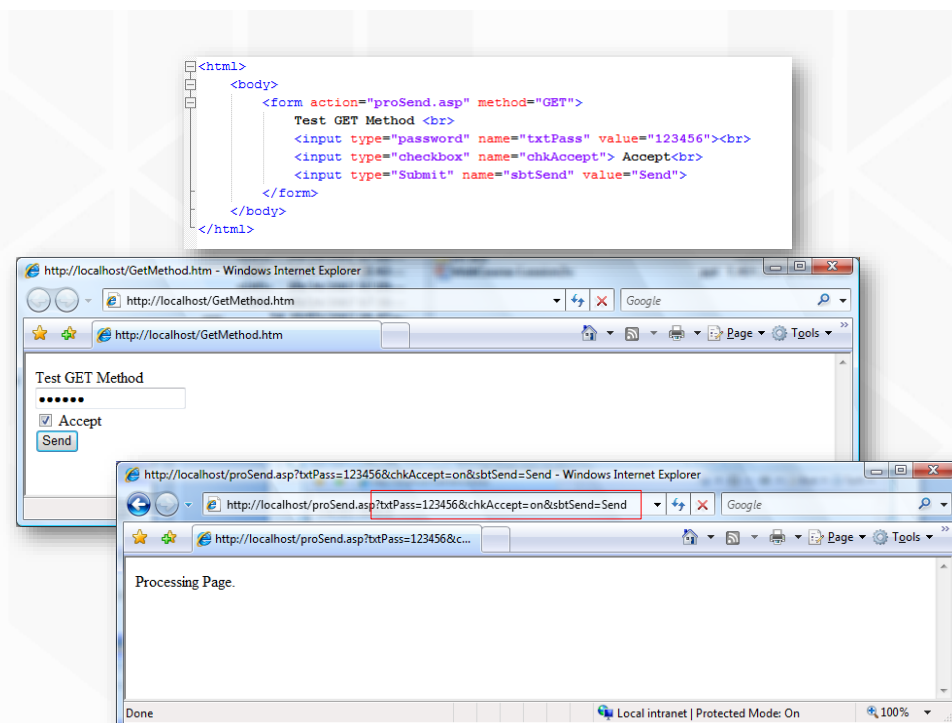


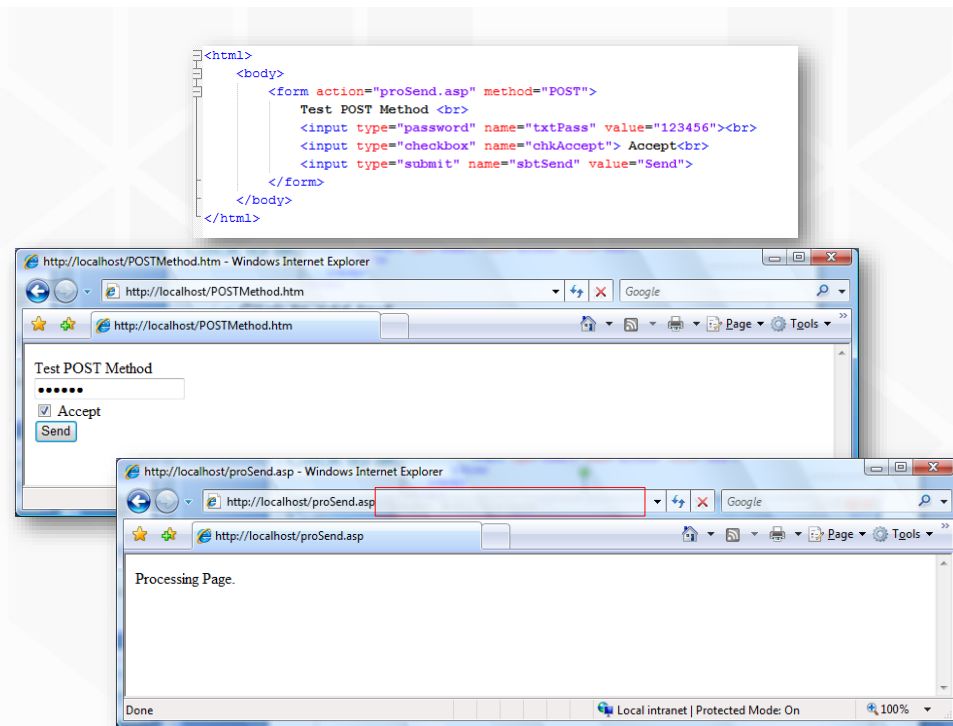
Phương thức GET

- ☐ Các đối số của Form được ghi kèm theo vào đường dẫn URL của thuộc tính Action trong tag <Form>
- ☐ Khối lượng dữ liệu đối số được truyền đi của Form bị giới hạn bởi chiều dài tối đa của một URL trên Address bar.
 - ☐ IE : Tối đa của một URL là 2.048 ký tự
 - ☐ Firefox : Tối thiểu của một URL là khoảng 100.000 ký tự
 - ☐ Safari : Tối thiểu của một URL là 80.000 ký tự
 - ☐ Opera : Tối thiểu của một URL là 190.000 ký tự
 - ☐ Apache Server : Tối đa của một URL là 8.192 ký tự
 - ☐ IIS Server : Tối đa của một URL là 16.384 ký tự

Phương thức POST

- ❑ Các đối số của Form được truyền “ngầm” bên dưới
- ❑ Khối lượng dữ liệu đối số được truyền đi của Form **không** phụ thuộc vào URL → Không bị giới hạn
- ❑ Chỉ sử dụng được phương thức truyền POST khi Action chỉ định đến trang web thuộc dạng trang web **có mã lệnh xử lý trên Server**





HTML Form và Validation

- ☐ HTML Form
- ☒ **Form Validation**

Data Validation

- ❑ **Data Validation** là quá trình xử lý để đảm bảo dữ liệu nhập từ người dùng được “sạch sẽ”, “chính xác” và “hữu ích”.
- ❑ Các loại validation thường gặp:
 - ❑ Required Fields
 - ❑ Date value
 - ❑ Numeric value
- ❑ Validation có thể được định nghĩa bằng nhiều phương thức và triển khai theo nhiều cách khác nhau
 - ❑ Validation phía server: xử lý sau khi dữ liệu được gửi về ⇨ **Bắt buộc!!!**
 - ❑ Validation phía client: xử lý trước khi dữ liệu được gửi đi

HTML Constraint Validation

- ❑ **HTML5** giới thiệu một khái niệm **HTML validation** mới được gọi là **constraint validation**.
- ❑ **HTML constraint validation** cơ bản dựa trên:
 - ❑ Constraint validation **HTML Input Attributes**
 - ❑ Constraint validation **CSS Pseudo Selectors**
 - ❑ Constraint validation **DOM Properties and Methods**

HTML5 Input Attributes

Thuộc tính	Ý nghĩa	Ví dụ minh họa
required	Bắt buộc phải nhập dữ liệu	<input required>
minlength	Số ký tự tối thiểu	<input minlength="3">
maxlength	Số ký tự tối đa	<input maxlength="8">
pattern	Giá trị phải khớp regex	<input pattern="[A-Za-z]{3,8}">
type	Kiểm tra kiểu dữ liệu (email, url, number, v.v.)	<input type="email">
min,max	Giới hạn giá trị số/ngày/thời gian	<input type="number" min="1" max="10">
step	Bước nhảy hợp lệ	<input type="number" step="2">

Ví dụ

```
<form action="#" method="GET" class="formT">
  <label for="txtEmail">Email:</label>
  <input type="text" id="txtEmail" required />
  <input type="submit" value="submit" />
</form>
```



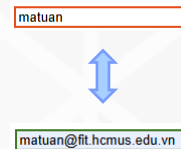
Email:

 Please fill out this field.

CSS Pseudo Selectors

- ❑ Sử dụng các pseudo-class như `:valid`, `:invalid`, `:required`, `:optional` để thay đổi giao diện khi dữ liệu hợp lệ hoặc không hợp lệ.

```
input:invalid {  
    border: 2px solid red;  
}  
input:valid {  
    border: 2px solid green;  
}
```



Constraint Validation DOM Methods

Phương thức	Chức năng chính
<code>checkValidity()</code>	Kiểm tra xem phần tử có thỏa mãn toàn bộ ràng buộc không. Trả về true nếu hợp lệ, ngược lại false .
<code>reportValidity()</code>	Tương tự <code>checkValidity()</code> , nhưng nếu không hợp lệ sẽ hiển thị thông báo lỗi mặc định của trình duyệt.
<code>setCustomValidity(message)</code>	Đặt thông báo lỗi tùy chỉnh cho phần tử. Nếu message là chuỗi rỗng, phần tử được đặt là hợp lệ. Nếu có chuỗi, phần tử sẽ bị coi là không hợp lệ và hiển thị thông báo này.

Ví dụ

```
<label for="numAge">Age:</label>
<input type="number" id="numAge" min="10" max="50" required />
<p id="errorMsg"></p>
<input type="button" value="OK" onclick="myValid()" />
```

```
function myValid() {
  const inpObj = document.getElementById('numAge');
  if (!inpObj.checkValidity()) {
    document.getElementById('errorMsg').innerHTML = inpObj.validationMessage;
  }
}
```



Age:

Please fill out this field.

OK

Age: 450

Value must be less than or equal to 50.

OK

Constraint Validation DOM Properties

Thuộc tính	Mô tả
validity	Trả về một đối tượng ValidityState chứa các thuộc tính boolean về trạng thái hợp lệ của phần tử (ví dụ: valueMissing , typeMismatch , patternMismatch , valid , v.v.).
validationMessage	Chuỗi thông báo lỗi sẽ được hiển thị nếu phần tử không hợp lệ. Nếu hợp lệ, sẽ là chuỗi rỗng.
willValidate	Boolean thể hiện phần tử có tham gia kiểm tra ràng buộc không (ví dụ: một số phần tử không được validate).

ValidityState

- ❑ **valueMissing**: **true** nếu trường bắt buộc nhưng chưa nhập.
- ❑ **typeMismatch**: **true** nếu giá trị không đúng kiểu quy định (ví dụ email không hợp lệ).
- ❑ **patternMismatch**: **true** nếu giá trị không khớp với biểu thức chính quy **pattern**.
- ❑ **rangeOverflow** & **rangeUnderflow**: đúng nếu vượt quá giới hạn **max** hoặc thấp hơn min.
- ❑ **stepMismatch**: đúng nếu giá trị không theo bước **step** quy định.
- ❑ **customError**: **true** nếu có lỗi tùy chỉnh bởi **setCustomValidity()**.
- ❑ **valid**: **true** nếu dữ liệu hợp lệ.

JavaScript Validation

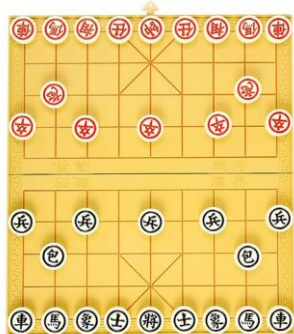
```
function valid() {  
    const inpAge = document.querySelector('#numAge');  
    let str = inpAge.value;  
    let eM = document.querySelector('#errorMsg');  
    if(str.length === 0) {  
        eM.innerHTML = "Can nhap tuoi";  
    }  
    else {  
        let age = parseInt(str);  
        if(age < inpAge.min) {  
            eM.innerHTML = `Tuoi phai >= ${inpAge.min}`;  
        }  
        if(age > inpAge.max) {  
            eM.innerHTML = `Tuoi phai <= ${inpAge.max}`;  
        }  
    }  
}
```

Nội dung

- ☐ *HTML5 Semantic*
- ☐ *Giới thiệu JavaScript*
- ☐ *JavaScript – cơ bản*
- ☐ *JavaScript – HTML DOM*
- ☐ *JavaScript – events*
- ☐ *HTML Form và Validation*
- ☐ **Bài tập**

Bài tập 1

- ☐ Dựng trang HTML là bàn cờ (cờ vua hoặc cờ tướng), cho phép sử dụng chuột để di chuyển các quân cờ (chỉ cho phép thay đổi vị trí vào ô đúng).



Bài tập 2



- ❑ Dùng HTML, CSS và Javascript để làm trang web như sau:

Bé tập tính

Số thứ nhất		☐ Cộng	☐ Trừ
Số thứ hai		☐ Nhân	☒ Chia
Kết quả		Tính	
Thông báo	Giá trị nhập ở ô <i>Số thứ nhất</i> không phải là số		

105

- ❑ Tạo giao diện, canh thẳng theo cột, chỉ có thể chọn một trong bốn phép tính.
- ❑ Viết mã Javascript để khi bấm vào nút tính, kết quả của phép tính đang chọn sẽ được điền vào ô kết quả.
- ❑ Vùng thông báo sẽ hiển thị thông báo lỗi khi
 - ❑ Di chuyển khỏi ô nhập *Số thứ nhất* hoặc *Số thứ hai* mà dữ liệu không phải là số thực.
 - ❑ Bấm nút *Tính* mà chưa chọn phép tính.
 - ❑ Chưa điền đủ hai số hợp lệ để thực hiện phép tính (4 điểm)

Bài tập 3

- ☐ Tạo trang HTML Đăng ký và Đăng nhập cho user với các thông tin như sau:
 - ☐ **Họ tên** (yêu cầu viết hoa chữ đầu).
 - ☐ **Username** (không được có khoảng trắng, chỉ gồm các ký tự, ký số và dấu _, không được bắt đầu bởi ký số).
 - ☐ **Email** (theo chuẩn email thông thường).
 - ☐ **Số phone** (theo chuẩn 10 số, bắt đầu bởi số 0).
 - ☐ **Ngày sinh** (theo định dạng dd/mm/yyyy và tuổi nằm trong khoảng [15, 55]).
- ☐ Yêu cầu:
 - ☐ Bố cục ngay giữa màn hình và các label, input,... phải ngay hàng thẳng lối.
 - ☐ Khi nhấn submit thì phải validate dữ liệu và thông báo chính xác lỗi cho người sử dụng.